TUTORIAL DE GIT

Capítulo 2. Fundamentos de Git

Juan Felipe Castañeda Rodríguez · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)
Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Este capítulo reúne los comandos que se usan el 90 % del tiempo: crear u obtener un repositorio, registrar cambios, revisar la historia, deshacer errores, trabajar con remotos y usar etiquetas.

2.1 Obtener un repositorio

Hay dos formas de tener un repositorio Git.

Inicializar uno en una carpeta existente:

```
cd mi_proyecto
git init
```

Esto crea el subdirectorio oculto .git con toda la estructura del repositorio. Todavía no se rastrea ningún archivo.

Clonar un repositorio existente:

```
git clone https://github.com/usuario/repositorio.git
git clone https://github.com/usuario/repositorio.git otra_carpeta
```

git clone descarga una copia completa, con toda la historia, y deja lista una copia de trabajo de la versión más reciente.

2.2 Guardar cambios en el repositorio

Cada archivo del directorio de trabajo está **rastreado** (Git lo conoce) o **sin rastrear** (Git nunca lo ha visto). Los rastreados pueden estar sin modificar, modificados o preparados.

Revisar el estado:

```
git status
git status -s
```

La versión corta (-s) muestra códigos: ?? sin rastrear, A añadido, M modificado.

Rastrear y preparar archivos:

```
git add README.md
git add .
```

git add es un comando multiuso: empieza a rastrear archivos nuevos, prepara archivos modificados y marca conflictos como resueltos. Si se modifica un archivo después de prepararlo, hay que volver a ejecutar git add, porque Git prepara el archivo tal como estaba en ese momento.

Ignorar archivos: en un archivo .gitignore se listan patrones de archivos que no se deben rastrear:

```
*.log
*.tmp
__pycache__/
```

```
.ipynb_checkpoints/
!importante.log
```

Ver las diferencias:

```
git diff
git diff --staged
```

El primero compara el directorio de trabajo con el área de preparación; el segundo compara lo preparado con el último commit.

Confirmar los cambios:

```
git commit -m "Agrega capítulo 2 del tutorial de Git"
git commit -a -m "Corrige ortografía en capítulo 1"
```

La opción -a prepara automáticamente todos los archivos ya rastreados que estén modificados, saltándose git add.

Buenas prácticas para mensajes de commit: una primera línea corta (unos 50 caracteres) en modo imperativo que diga qué hace el cambio ("Agrega", "Corrige", "Actualiza"), y si hace falta, una línea en blanco y un párrafo que explique el porqué.

Eliminar y mover archivos:

```
git rm archivo.txt
git rm --cached archivo.txt
git mv nombre_viejo.txt nombre_nuevo.txt
```

--cached deja de rastrear el archivo pero lo conserva en el disco.

2.3 Ver el historial de confirmaciones

```
git log
git log -p -2
git log --stat
git log --oneline --graph --all
git log --since=2.weeks --author="Juan"
git log --pretty=format:"%h - %an, %ar : %s"
```

- -p muestra las diferencias introducidas en cada commit.
- --stat muestra estadísticas de archivos modificados.
- --oneline --graph dibuja la historia compacta con las ramas.
- --since, --until y --author filtran resultados.

2.4 Deshacer cosas

Rehacer el último commit (por ejemplo, si se olvidó un archivo o hay un error en el mensaje):

```
git commit --amend
```

Sacar un archivo del área de preparación:

```
git restore --staged archivo.txt
git reset HEAD archivo.txt
```

Descartar cambios en el directorio de trabajo (cuidado: se pierden los cambios no confirmados):

```
git restore archivo.txt
git checkout -- archivo.txt
```

2.5 Trabajar con remotos

Un remoto es una versión del proyecto alojada en otro lugar (por ejemplo, GitHub). Al clonar, Git crea automáticamente el remoto origin.

```
git remote -v
git remote add origin https://github.com/usuario/repo.git
git fetch origin
git pull
git push origin main
git remote show origin
git remote rename origin principal
git remote remove principal
```

- **fetch** descarga los datos nuevos del remoto sin mezclarlos con tu trabajo.
- **pull** descarga y fusiona automáticamente en la rama actual.
- **push** envía tus commits al remoto; solo funciona si tienes permisos y nadie empujó antes que tú (si no, primero hay que traer los cambios).

2.6 Etiquetado

Las etiquetas (tags) marcan puntos importantes de la historia, como versiones publicadas.

```
git tag
git tag -a v1.0 -m "Primera versión del tutorial"
git tag v1.0-ligera
git show v1.0
git push origin v1.0
git push origin --tags
```

Las etiquetas **anotadas** (-a) guardan autor, fecha y mensaje; las **ligeras** son solo un puntero a un commit. Las etiquetas no se envían con git push normal: hay que empujarlas explícitamente.

2.7 Alias

Los alias permiten crear atajos para comandos largos:

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.ultimo "log -1 HEAD"
```

2.8 Resumen

Con lo visto ya se puede crear o clonar un repositorio, preparar y confirmar cambios, revisar el historial, corregir errores, sincronizar con GitHub y marcar versiones. Este es exactamente el flujo usado en esta actividad: git add, git commit y git push por cada capítulo.

Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2